

Blackthorn

Security Review For **Dromos**

Collaborative Audit Prepared For: **Dromos**

Lead Security Expert(s): **bughuntoor**

hash

xiaoming90

Date Audited: **July 20 - August 17, 2026**

Introduction

MetaDEX03 is Dromos Labs' next-generation evolution of the liquidity coordination model pioneered through Velodrome and Aerodrome. It builds on the core ideas that made those systems successful while rethinking the architecture for a more flexible, composable, and multi-chain future.

The protocol brings together liquidity, governance, emissions, and incentives into a single coordinated system, while introducing a number of major improvements intended to make participation more dynamic and the protocol easier to extend. MetaDEX03 moves beyond the rigid epoch-driven structure of earlier implementations, expands the model across multiple chains, and introduces new infrastructure for routing user actions, managing assets, and supporting more modular AMM behavior.

Scope

Repository: [dromos-labs/metadex](https://github.com/dromos-labs/metadex)

Audited Commit: [f91e1624c92a86b3221a4636d4341e4f5d1afd54](#)

Final Commit: [47e45eb40553b7cf6190301fe7099e549c171890](#)

Files:

- [src/factories/FactoryRegistry.sol](#)
- [src/factories/PoolFactory.sol](#)
- [src/factories/StablePoolFactory.sol](#)
- [src/factories/VolatilePoolFactory.sol](#)
- [src/fees/CustomFeeModule.sol](#)
- [src/fees/DiscountRegistry.sol](#)
- [src/pools/PoolFees.sol](#)
- [src/pools/Pool.sol](#)
- [src/pools/StablePool.sol](#)

- src/pools/tape/BasePoolTape.sol
- src/pools/tape/CLPoolTape.sol
- src/pools/tape/PoolTape.sol
- src/pools/VolatilePool.sol
- V3/src/access/GuardedAccessControlEnumerable.sol
- V3/src/art/BokkyPooBahsDateTimeLibrary.sol
- V3/src/art/VeArtProxy.sol
- V3/src/bridge/hyperlane/HyperlaneAdapter.sol
- V3/src/bridge/LeafMessageOrchestrator.sol
- V3/src/bridge/MessageOrchestrator.sol
- V3/src/bridge/RootLocalAdapter.sol
- V3/src/bridge/RootMessageOrchestrator.sol
- V3/src/core/VotingEscrow.sol
- V3/src/factories/GaugeFactory.sol
- V3/src/gauges/Gauge.sol
- V3/src/gauges/V2Gauge.sol
- V3/src/handlers/LeafEmissionsHandler.sol
- V3/src/handlers/RootEmissionsHandler.sol
- V3/src/indexation/PoolFactoryIndexation.sol
- V3/src/interfaces/access/IGuardedAccessControlEnumerable.sol
- V3/src/interfaces/art/IVeArtProxy.sol
- V3/src/interfaces/bridge/hyperlane/IHyperlaneAdapter.sol
- V3/src/interfaces/bridge/ILeafMessageOrchestrator.sol
- V3/src/interfaces/bridge/IMessageAdapter.sol

- V3/src/interfaces/bridge/IMessageOrchestrator.sol
- V3/src/interfaces/bridge/IRootLocalAdapter.sol
- V3/src/interfaces/bridge/IRootMessageOrchestrator.sol
- V3/src/interfaces/core/IVotingEscrow.sol
- V3/src/interfaces/external/ITokenRegistry.sol
- V3/src/interfaces/external/IWETH.sol
- V3/src/interfaces/factories/IGaugeFactory.sol
- V3/src/interfaces/gauges/IGauge.sol
- V3/src/interfaces/gauges/IV2Gauge.sol
- V3/src/interfaces/handlers/IEmissionsHandler.sol
- V3/src/interfaces/handlers/ILeafEmissionsHandler.sol
- V3/src/interfaces/indexation/IPoolFactoryIndexation.sol
- V3/src/interfaces/minter/IDynamicRateModule.sol
- V3/src/interfaces/minter/IMinter.sol
- V3/src/interfaces/minter/IToken.sol
- V3/src/interfaces/pools/IPool.sol
- V3/src/interfaces/rewards/IFeeDistribution.sol
- V3/src/interfaces/rewards/IncentiveStreaming.sol
- V3/src/interfaces/rewards/IVotingCheckpoints.sol
- V3/src/interfaces/rewards/IVotingRewardsFactory.sol
- V3/src/interfaces/rewards/IVotingRewardsManager.sol
- V3/src/interfaces/.solhint.json
- V3/src/interfaces/splitter/ISplitter.sol
- V3/src/interfaces/tmp/ITempFactoryRegistry.sol

- V3/src/interfaces/tmp/ITempGauge.sol
- V3/src/interfaces/tmp/ITempReward.sol
- V3/src/interfaces/token/IReceiptTokenExtensions.sol
- V3/src/interfaces/token/ITokenExtensions.sol
- V3/src/interfaces/voter/IGasGuard.sol
- V3/src/interfaces/voter/ILeafVoter.sol
- V3/src/interfaces/voter/IVoterCommon.sol
- V3/src/interfaces/voter/IVoter.sol
- V3/src/interfaces/vpm/IVoterPaymentsModule.sol
- V3/src/libraries/BalanceLogicLibrary.sol
- V3/src/libraries/CheckpointLogicLibrary.sol
- V3/src/libraries/DelegationLogicLibrary.sol
- V3/src/libraries/ParamsLib.sol
- V3/src/libraries/ProtocolConstants.sol
- V3/src/libraries/Roles.sol
- V3/src/libraries/SafeCastLibrary.sol
- V3/src/libraries/VelodromeTimeLibrary.sol
- V3/src/minter/Minter.sol
- V3/src/rewards/FeeDistribution.sol
- V3/src/rewards/IncentiveStreaming.sol
- V3/src/rewards/VotingCheckpoints.sol
- V3/src/rewards/VotingRewardsFactory.sol
- V3/src/rewards/VotingRewardsManager.sol
- V3/src/splitter/Splitter.sol

- V3/src/token/ReceiptToken.sol
 - V3/src/token/Token.sol
 - V3/src/voter/GasGuard.sol
 - V3/src/voter/LeafVoter.sol
 - V3/src/voter/Voter.sol
 - V3/src/vpm/VoterPaymentsModule.sol
-

Repository: dromos-labs/metadex-slipstream

Audited Commit: 00fc7d944ca30472ebcb1622343c4df7e2a73c1a

Final Commit: 249fc43d345e72b3f88a937dcecdb4513721c064

Files:

- contracts/core/CLFactory.sol
- contracts/core/CLPool.sol
- contracts/core/fees/CustomSwapFeeModule.sol
- contracts/core/fees/CustomUnstakedFeeModule.sol
- contracts/core/fees/DynamicSwapFeeModule.sol
- contracts/core/fees/README.md
- contracts/core/indexation/CLFactoryIndexation.sol
- contracts/core/interfaces/callback/ICLFlashCallback.sol
- contracts/core/interfaces/callback/ICLMintCallback.sol
- contracts/core/interfaces/callback/ICLSwapCallback.sol
- contracts/core/interfaces/fees/ICustomFeeModule.sol
- contracts/core/interfaces/fees/IDiscountRegistry.sol
- contracts/core/interfaces/fees/IDynamicFeeModule.sol

- `contracts/core/interfaces/fees/IFeeModule.sol`
- `contracts/core/interfaces/ICLFactory.sol`
- `contracts/core/interfaces/ICLPool.sol`
- `contracts/core/interfaces/IERC20Minimal.sol`
- `contracts/core/interfaces/IFactoryRegistry.sol`
- `contracts/core/interfaces/IMinter.sol`
- `contracts/core/interfaces/indexation/ICLFactoryIndexation.sol`
- `contracts/core/interfaces/IPoolFactory.sol`
- `contracts/core/interfaces/IPool.sol`
- `contracts/core/interfaces/IVoter.sol`
- `contracts/core/interfaces/IVotingEscrow.sol`
- `contracts/core/interfaces/LICENSE`
- `contracts/core/interfaces/pool/ICLPoolActions.sol`
- `contracts/core/interfaces/pool/ICLPoolConstants.sol`
- `contracts/core/interfaces/pool/ICLPoolDerivedState.sol`
- `contracts/core/interfaces/pool/ICLPoolEvents.sol`
- `contracts/core/interfaces/pool/ICLPoolOwnerActions.sol`
- `contracts/core/interfaces/pool/ICLPoolState.sol`
- `contracts/core/interfaces/tape/IBasePoolTape.sol`
- `contracts/core/libraries/BitMath.sol`
- `contracts/core/libraries/FixedPoint128.sol`
- `contracts/core/libraries/FixedPoint96.sol`
- `contracts/core/libraries/FullMath.sol`
- `contracts/core/libraries/LICENSE`

- `contracts/core/libraries/LICENSE_MIT`
- `contracts/core/libraries/LiquidityMath.sol`
- `contracts/core/libraries/LowGasSafeMath.sol`
- `contracts/core/libraries/Oracle.sol`
- `contracts/core/libraries/Position.sol`
- `contracts/core/libraries/SafeCast.sol`
- `contracts/core/libraries/SqrtPriceMath.sol`
- `contracts/core/libraries/SwapMath.sol`
- `contracts/core/libraries/TickBitmap.sol`
- `contracts/core/libraries/TickMath.sol`
- `contracts/core/libraries/Tick.sol`
- `contracts/core/libraries/TransferHelper.sol`
- `contracts/core/libraries/UnsafeMath.sol`
- `contracts/core/LICENSE`
- `contracts/core/test/BitMathEchidnaTest.sol`
- `contracts/core/test/BitMathTest.sol`
- `contracts/core/test/CLPoolSwapTest.sol`
- `contracts/core/test/CoreTestERC20.sol`
- `contracts/core/test/Create2Address.sol`
- `contracts/core/test/FullMathEchidnaTest.sol`
- `contracts/core/test/FullMathTest.sol`
- `contracts/core/test/LiquidityMathTest.sol`
- `contracts/core/test/LowGasSafeMathEchidnaTest.sol`
- `contracts/core/test/MockTimeCLPool.sol`

- `contracts/core/test/OracleEchidnaTest.sol`
- `contracts/core/test/OracleTest.sol`
- `contracts/core/test/SqrtPriceMathEchidnaTest.sol`
- `contracts/core/test/SqrtPriceMathTest.sol`
- `contracts/core/test/SwapMathEchidnaTest.sol`
- `contracts/core/test/SwapMathTest.sol`
- `contracts/core/test/TestCLCallee.sol`
- `contracts/core/test/TestCLReentrantCallee.sol`
- `contracts/core/test/TestCLRouter.sol`
- `contracts/core/test/TestCLSwapPay.sol`
- `contracts/core/test/TickBitmapEchidnaTest.sol`
- `contracts/core/test/TickBitmapTest.sol`
- `contracts/core/test/TickEchidnaTest.sol`
- `contracts/core/test/TickMathEchidnaTest.sol`
- `contracts/core/test/TickMathTest.sol`
- `contracts/core/test/TickOverflowSafetyEchidnaTest.sol`
- `contracts/core/test/TickTest.sol`
- `contracts/core/test/UnsafeMathEchidnaTest.sol`
- `contracts/gauge/CLGaugeFactory.sol`
- `contracts/gauge/CLGauge.sol`
- `contracts/gauge/Gauge.sol`
- `contracts/gauge/interfaces/ICLGaugeFactory.sol`
- `contracts/gauge/interfaces/ICLGauge.sol`
- `contracts/gauge/interfaces/ICLPool.sol`

- `contracts/gauge/interfaces/IGauge.sol`
- `contracts/gauge/interfaces/ILeafVoter.sol`
- `contracts/gauge/interfaces/INonfungiblePositionManager.sol`
- `contracts/gauge/interfaces/IReward.sol`
- `contracts/gauge/interfaces/IVotingRewardsManager.sol`
- `contracts/libraries/EnumerableSet.sol`
- `contracts/libraries/VelodromeTimeLibrary.sol`
- `contracts/periphery/base/BlockTimestamp.sol`
- `contracts/periphery/base/ERC721Permit.sol`
- `contracts/periphery/base/LiquidityManagement.sol`
- `contracts/periphery/base/Multicall.sol`
- `contracts/periphery/base/PeripheryImmutableState.sol`
- `contracts/periphery/base/PeripheryPayments.sol`
- `contracts/periphery/base/PeripheryPaymentsWithFee.sol`
- `contracts/periphery/base/PeripheryValidation.sol`
- `contracts/periphery/base/SelfPermit.sol`
- `contracts/periphery/examples/PairFlash.sol`
- `contracts/periphery/interfaces/external/IERC1271.sol`
- `contracts/periphery/interfaces/external/IERC20PermitAllowed.sol`
- `contracts/periphery/interfaces/external/IWETH9.sol`
- `contracts/periphery/interfaces/IERC20Metadata.sol`
- `contracts/periphery/interfaces/IERC4906.sol`
- `contracts/periphery/interfaces/IERC721Permit.sol`
- `contracts/periphery/interfaces/ILpMigrator.sol`

- `contracts/periphery/interfaces/IMixedRouteQuoterV1.sol`
- `contracts/periphery/interfaces/IMixedRouteQuoterV2.sol`
- `contracts/periphery/interfaces/IMulticall.sol`
- `contracts/periphery/interfaces/INonfungiblePositionManager.sol`
- `contracts/periphery/interfaces/INonfungibleTokenPositionDescriptor.sol`
- `contracts/periphery/interfaces/IPeripheryImmutableState.sol`
- `contracts/periphery/interfaces/IPeripheryPayments.sol`
- `contracts/periphery/interfaces/IPeripheryPaymentsWithFee.sol`
- `contracts/periphery/interfaces/IQuoter.sol`
- `contracts/periphery/interfaces/IQuoterV2.sol`
- `contracts/periphery/interfaces/ISelfPermit.sol`
- `contracts/periphery/interfaces/ISwapRouter.sol`
- `contracts/periphery/interfaces/ITickLens.sol`
- `contracts/periphery/lens/CLInterfaceMulticall.sol`
- `contracts/periphery/lens/MixedRouteQuoterV1.sol`
- `contracts/periphery/lens/MixedRouteQuoterV2.sol`
- `contracts/periphery/lens/MixedRouteQuoterV3.sol`
- `contracts/periphery/lens/Quoter.sol`
- `contracts/periphery/lens/QuoterV2.sol`
- `contracts/periphery/lens/README.md`
- `contracts/periphery/lens/TickLens.sol`
- `contracts/periphery/libraries/BytesLib.sol`
- `contracts/periphery/libraries/CallbackValidation.sol`
- `contracts/periphery/libraries/ChainId.sol`

- `contracts/periphery/libraries/HexStrings.sol`
- `contracts/periphery/libraries/LiquidityAmounts.sol`
- `contracts/periphery/libraries/NFTDescriptor.sol`
- `contracts/periphery/libraries/NFTSVG.sol`
- `contracts/periphery/libraries/OracleLibrary.sol`
- `contracts/periphery/libraries/Path.sol`
- `contracts/periphery/libraries/PoolAddress.sol`
- `contracts/periphery/libraries/PoolTicksCounter.sol`
- `contracts/periphery/libraries/PositionKey.sol`
- `contracts/periphery/libraries/PositionValue.sol`
- `contracts/periphery/libraries/SqrtPriceMathPartial.sol`
- `contracts/periphery/libraries/TokenRatioSortOrder.sol`
- `contracts/periphery/libraries/TransferHelper.sol`
- `contracts/periphery/LICENSE`
- `contracts/periphery/LpMigrator.sol`
- `contracts/periphery/NonfungiblePositionManager.sol`
- `contracts/periphery/NonfungibleTokenPositionDescriptor.sol`
- `contracts/periphery/SwapRouter.sol`
- `contracts/periphery/test/Base64Test.sol`
- `contracts/periphery/test/LiquidityAmountsTest.sol`
- `contracts/periphery/test/MockObservable.sol`
- `contracts/periphery/test/MockObservations.sol`
- `contracts/periphery/test/MockTimeNonfungiblePositionManager.sol`
- `contracts/periphery/test/MockTimeSwapRouter.sol`

- `contracts/periphery/test/NFTDescriptorTest.sol`
- `contracts/periphery/test/NFTManagerCallee.sol`
- `contracts/periphery/test/NonfungiblePositionManagerPositionsGasTest.sol`
- `contracts/periphery/test/OracleLibraryTest.sol`
- `contracts/periphery/test/PathTest.sol`
- `contracts/periphery/test/PeripheryImmutableStateTest.sol`
- `contracts/periphery/test/PoolTicksCounterTest.sol`
- `contracts/periphery/test/PositionValueTest.sol`
- `contracts/periphery/test/SelfPermitTest.sol`
- `contracts/periphery/test/TestCallbackValidation.sol`
- `contracts/periphery/test/TestERC20Metadata.sol`
- `contracts/periphery/test/TestERC20PermitAllowed.sol`
- `contracts/periphery/test/TestERC20.sol`
- `contracts/periphery/test/TestMulticall.sol`
- `contracts/periphery/test/TestPositionNFTOwner.sol`
- `contracts/periphery/test/TickLensTest.sol`

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or

related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
3	14	6

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@bughuntoor

A stylized, cursive handwritten signature in white ink, featuring a large, sweeping 'D' shape that loops around the text.

@hash

A stylized, cursive handwritten signature in white ink, featuring a large, sweeping 'A' shape that loops around the text.

@xiaoming90

A stylized, cursive handwritten signature in white ink, featuring a large, sweeping 'X' shape that loops around the text.

Issue H-1: Stale Leaf Emission Scalar Causes Unbacked xTOKEN Minting and Redemption Loss [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/150>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

The new PR404 uses a global scalar (`emissionsPerVP`). This global scalar is recomputed on the root chain for most users' actions. However, the leaf chain only receives a new global scalar if it is a cross-chain allocation message that affects the chain. As such, if no one voted on a specific leaf chain for a while, the leaf chain will accrue at the old, too-high scalar, while the root accrues at a new, lower one.

Assume that at T0, the emission rate is 1 TOKEN/s, and Chain A holds 100 weight, and the total weight is 1000. At T1, the total weight grows from 1000 to 10,000.

The state is as follows:

- Leaf (frozen at T0) - Scalar = $1e18 \cdot 1e18 / 1000e18 = 1e15$ (0.1 TOKEN per second to Chain A)
- Root (live at T1) - Scalar = $1e18 \cdot 1e18 / 10000e18 = 1e14$ (0.01 TOKEN per second to Chain A)

Over one quiet week (604800s), with Chain A receiving no messages:

- Leaf accrues $0.1 \times 604800 = 60,480$ TOKEN into Chain A's gauge ceilings
- Root accrues $0.01 \times 604800 = 6,048$ TOKEN into Chain A's `chainStates[A].ceiling`
- Deficit = 54,432 TOKEN

Gauges on Chain A can now mint the full 60,480 as xTOKEN. When LPs redeem, the first 6,048 TOKEN will succeed. Past that, redeem burns the receipt on the leaf and only then

dispatches, so root's CeilingExceeded fires on delivery, after the burn has already been committed. So, the LP loses the xTOKEN and receives nothing.

Impact

Leaf chains can over-accrue gauge ceilings using a stale emission scalar, allowing gauges to mint unbacked xTOKEN. When users redeem beyond the root chain's lower ceiling, their xTOKEN is permanently burned while the root rejects the redemption, leaving them with no TOKEN.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/Voter.sol#L1316-L1323>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L979-L982>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L299-L311>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/Voter.sol#L412-L419>

Tool Used

Manual Review

Recommendation

Keep leaf-chain emission accounting synchronized with the authoritative root state, and ensure xTOKEN cannot be irreversibly burned unless sufficient root-chain backing is confirmed. Prevent stale ceilings from authorizing unbacked minting.

Discussion

agusduha

This drift is acknowledge, it is a design decision to avoid needing a message to settling every chain each time the total weight changes. We can have a mechanism in place to update leaves easily with an alloc chain msg when they cross certain drift threshold.

Issue H-2: Out-of-Order Messages Create Permanent Phantom Weight [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/151>

Vulnerability Detail

Within `applyChainAllocation`, the `_refreshShape` ensures that only the latest shape can write to `latestTokenSnapshot[_tokenId]`, and shape in stale message is discarded. In contrast, the `allocationDelta` in either latest or stale messages are applied exactly once no matter when it lands because the design deemed it as additive. In an edge case, these two properties can lead to phantom weight. Consider the following scenario: Let `B` be the next week boundary, `L` a registered, active remote chain, and `X = 1,000,000e18 TOKEN`.

1. Bob executes `createStake(X, 1, false)` 2 seconds before `B`
2. Bob executes `allocateChains(L, +X)`. A new `AllocateChain` message that contains `{allocationDelta: X, shape {X, B}}` with `nonce=100` is generated and submitted. No one relay this message for a while due to high gas, network congestion or not properly funded or other reasons. Anyway, Hyperlane does not guarantee messages are delivered in sequence or FIFO basis in the first place.
3. Meanwhile, Bob executes `withdraw(T)` at or after `B`. His `veNFT`'s stake record becomes `{amount: 0, end: 0, isPermanent: false}`.
4. Bob executes `allocateGauges(T, L, [])`, with empty list. A new `AllocateGauge` message that contains `{allocations [], shape {0,0}}` with `nonce=101` is generated and submitted.
5. The `allocateGauges` message get relayed to LEAF `L` first rather than the `allocateChains` message. When handling the `allocateGauges` message, the `lastShapeNonce[T]` will be updated 101. The `latestTokenSnapshot[T]` remain at `{0,0}`.
6. Subsequently, the `AllocateChain` gets handled. The `_refreshShape` will be `false` because `(100 > lastShapeNonce[T]) = (100 > 101) = FALSE`. So, the `AllocateChain`

message's $\{X, B\}$ shape is discarded, and the stored $\{0, 0\}$ shape is used instead.

7. The `allocationDelta` is additive per design, so the `AllocateChain`'s `allocationDelta` will proceed to get applied, and `gaugeStates[ZERO_GAUGE].point.permanentStakeBalance += X`.
8. So, the leaf books the amount as a permanent (never-decaying) position that the root has already deleted from its own books, and it permanently consumes the chain's redemption headroom.
9. As a result, redemptions can burn receipt tokens on the leaf and initially revert with `CeilingExceeded` on root. Although those messages remain retryable, the phantom weight creates a permanent aggregate redemption-capacity deficit.

Impact

Out-of-order delivery can apply a stale allocation delta against a newer zeroed stake shape, creating permanent phantom weight on the leaf. This reduces redemption capacity and can repeatedly block otherwise valid redemptions with `CeilingExceeded`.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/bridge/LeafMessageOrchestrator.sol#L218-L219>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/bridge/LeafMessageOrchestrator.sol#L256-L257>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L341-L380>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L1032-L1043>

Tool Used

Manual Review

Recommendation

Bind each allocation change to the stake state that produced it, and safely reconcile, never discard, changes whose stake state has been superseded, since root has already booked them. Ensure out-of-order delivery cannot book stale allocations as live or permanent voting weight.

Discussion

Oxng

Fixed here: <https://github.com/dromos-labs/metadex/pull/482>

Issue H-3: User deposit will be lost in case the relay is closed in the same block [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/160>

Vulnerability Detail

Pending user deposits can only be processed in case the requestedAt timestamp is less than the lastAllocation timestamp

```
if (_entry.requestedAt >= _ctx.lastAllocation) break;
```

When the relay is closed, lastAllocation is permanently set to block.timestamp since further invocations of allocate will revert

```
function close() external nonReentrant {
    if (closed) revert RelayClosed();
    if (!hasRole(KEEPER, msg.sender)) {
        AllocationLib.requireUncoveredHead(withdrawRing, withdrawals, VOTER,
            ↪ _relayConfig.tokenId);
    }

    closed = true;
    lastAllocation = uint48(block.timestamp);
}
```

This makes it impossible for the deposits that happened at the same timestamp to be processed causing the users to lose their deposits

Impact

Users will lose their deposit amount permanently

Code Snippet

Tool Used

Manual Review

Recommendation

Allow a wildcard scenario for processing user deposits when the relay has been closed

Discussion

0xng

Fixed here: <https://github.com/dromos-labs/metadex/pull/487/changes/b24f4397cd9ebb18ed71c18d3d9b3b31d3bb8004>

10xhash

lgtn

Issue M-1: `_segmentAccrual` applies a single average weight across an interval with variable `emissionsPerVP`, making chain ceilings higher than emitted supply [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/142>

Vulnerability Detail

Ideally the amount of emission allocated to a chain for a segment $[t_0, t_1]$ should be $\int_{t_0}^{t_1} w(t) \cdot s(t) dt$, where w represents the voting power for the chain and s represents the emission per voting power. The `_segmentAccrual` function deviates from this ideal and instead applies the average weight of the chain ie. $(startWeight + endWeight) / 2$ throughout the `_indexDelta` ie. $\bar{w} \cdot s dt$. The implementation also simplifies the $s dt$ part by using the latest computed total voting power for the denominator rather than the instantaneous voting power. This change is guaranteed to make the summation $\leq$ when compared to the ideal. But using the arithmetic mean ie. $\bar{w}$ can cause the chain's accrual to be higher than the actual emission in case `emissionsPerVP` gets updated in between

Eg:

```
Emission rate r = 1 token/sec. One chain holds all the voting power, so it should
↳ earn every token emitted. Its weight decays 100 → 1 over 100 one-second ticks.
↳ emissionsPerVP = r / W is resampled each tick.
```

```
Settled every tick (correct)
```

```
Each tick accrues  $W_i \times (1/W_i) \times 1s = 1$  token:
```

```
 $100 \times 1/100 + 99 \times 1/99 + \dots + 1 \times 1/1 = 1 + 1 + \dots + 1 = 100$  tokens
```

```
Settled once at the end (what the code does)
```

The index still integrates the same per-tick scalars:

$$\Delta_{\text{index}} = 1/100 + 1/99 + \dots + 1/1 = H \approx 5.187$$

but the weight is collapsed to its endpoint average:

$$\begin{aligned} \text{avg}(W) &= (100 + 1) / 2 = 50.5 \\ \text{accrual} &= 50.5 \times 5.187 \approx 262 \text{ tokens} \end{aligned}$$

Impact

Σ ceilings can drift above the minter's schedule

Code Snippet

- `Voter._segmentAccrual`

Tool Used

Manual Review

Recommendation

Prefer under-accrual over over-accrual by using the chain's `endWeight` rather than the arithmetic mean

Discussion

0xng

Fixed here: <https://github.com/dromos-labs/metadex/pull/486>

10xhash

`_settleGlobalIndex` can be invoked inside `registerChain` so that the indexes and timestamp of the chain are in sync

0xng

Good call, added here: <https://github.com/dromos-labs/metadex/pull/534>

Issue M-2: PT holders might be indefinitely locked out of their funds [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/144>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

PT holders might be indefinitely locked out of their funds.

Vulnerability Detail

The new relay contracts introduce the idea of PT (Principal Tokens) and YT (Yield tokens). The yield tokens receive the yield earned from the assets in perpetuity, while the PT tokens represent ownership of the principal.

The problem arises from the fact, that unlike the common Pendle-style PT/YT tokens, where they have a maturity date past which the PT tokens can be redeemed on their own, here tokens can only be redeemed if the user holds both the amount in PT tokens and in YT tokens.

This can theoretically create a weird dynamic, where YT token holder, decides to "hold hostage" the PT holder's assets, by simply not selling their YT tokens indefinitely. The same can also be done by the PT holder, but the difference is that the YT holder will be earning yield in perpetuity, while the PT holder will not be receiving any assets.

Impact

Users might be locked out of their funds indefinitely.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider switching to maturity-based PT/YT tokens.

Discussion

agusduha

Intended https://dromoslabs-group.slack.com/archives/C0BCBSDQEQ7/p1787582575457869?thread_ts=1786709706.150929cid=C0BCBSDQEQ7

Issue M-3: Failed Settlement Lets New Stakers Capture Past Emissions [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/146>

Vulnerability Detail

The following is the new code introduced:

```
/// @dev Realizes rewards up to the current block. If the cached active rate has
↪ expired, pulls a fresh rate from the gauge.
/// Calls {_updateRewardsGrowthGlobal} to update the rewards to distribute and
↪ lastUpdated.
function _settleToBlock() internal {
    uint48 _timestamp = uint48(block.timestamp);
    if (_timestamp == lastUpdated) return;

    address _gauge = gauge;
    if (_gauge == address(0)) return;

    if (_timestamp > activeRateValidUntil) {
        try ICLGauge(_gauge).settleGauge() returns (uint128 rate, uint48 from, uint48
↪ to) {
            _settleRate(rate, from, to);
        } catch {
            // activeRate is realized only through activeRateValidUntil and
↪ activeRateValidUntil is not updated
            _updateRewardsGrowthGlobal(uint256(activeRate) * (activeRateValidUntil -
↪ lastUpdated), activeRateValidUntil);
        }
    } else {
        _updateRewardsGrowthGlobal(uint256(activeRate) * (_timestamp - lastUpdated),
↪ _timestamp);
    }
}
```

In the Phase 2 code, if the `ICLGauge(_gauge).settleGauge()` fails, it will go to the catch block and realize until the `activeRateValidUntil`.

However, this approach silently gives up on paying out the elapsed time, but still lets the caller change who is staked. The skipped emissions are paid out later, to whoever happens to be staked then.

Consider the following setup.

- Emission rate = 10 TOKEN/sec
- Settled through $t = 100$, with the cached window ending there too (`lastUpdated == activeRateValidUntil == 100`)
- Nothing is staked in range (`stakedLiquidity = 0`)

At $T=200$, a single LP deposits into the gauge and its position is in range.

At $T=201$, a settlement succeeds and everything is brought current.

Consider the following two scenarios:

Scenario 1: Settlements at $T=200$ works

The `stake()` settle first. At this point, the `stakedLiquidity` is zero so the 1000 TOKEN accrued over $[100, 200)$ will go to the rollover, and returned to the LeafVoter as unused emission. Then, the stake lands.

At $T=201$, the LP is credited for one second he was actually staked.

Scenario 2: Settlement at $T=200$ fails

The catch credits nothing and pins `lastUpdated` at 100. The stake lands anyway, and the LP's baseline is snapshotted from the stale accumulator.

At $t = 201$ the recovery window $(10, 100, 201)$ arrives and 1,010 TOKEN are credited in one lump, dividing by a `stakedLiquidity` that is now entirely this one LP's.

Following is a comparison of where the 1010 TOKEN of emissions end up in the above two scenarios:

	Settlement OK	Settlement Failed
Paid to the LP who deposited at $t = 200$	10	
<code>rollover</code> => returned to LeafVoter as unused emissions (on the next push)	1,000	
Total emitted	1,010	

The current design breaks the "settle-before-mutate" invariant, and thus lead to the above issue. The invariant is that every code path that changes who is staked or which range is in force (a tick-crossing swap) must first bring the accumulator up to the present moment before allowing it to change the state.

Although the functions attempt to settle first before mutate by calling the `_settleToBlock()`, the issue is that settle is best-effort while the mutation is unconditional.

Impact

If gauge settlement fails before a stake or liquidity mutation, the mutation proceeds against a stale reward accumulator. A later successful settlement can allocate emissions accrued before the participant joined to the new staker, diverting unused emissions from the LeafVoter and unfairly redistributing rewards.

Code Snippet

<https://github.com/dromos-labs/metadex-slipstream/blob/231bc3234a1eb14cb362e29918f5bca5a9c65ad9/contracts/core/CLPool.sol#L1028-L1045>

<https://github.com/dromos-labs/metadex-slipstream/blob/231bc3234a1eb14cb362e29918f5bca5a9c65ad9/contracts/core/CLPool.sol#L592-L598>

<https://github.com/dromos-labs/metadex-slipstream/blob/231bc3234a1eb14cb362e29918f5bca5a9c65ad9/contracts/core/CLPool.sol#L801-L806>

Tool Used

Manual Review

Recommendation

Require rewards to be fully settled before any action changes staked liquidity or the active range. If settlement cannot complete, prevent the mutation or preserve deferred emissions separately so they cannot be distributed according to the later stake state.

Discussion

agusduha

This finding is outdated after the voting redesign

rhokterhus

I agree fixed now, specifically

<https://github.com/dromos-labs/metadex-slipstream/pull/33>

Issue M-4: Restricted veNFT Value Can Bypass Migration Controls Through Token-ID Changes [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/147>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

The restricted mapping in the migration contract is to block specified token ID from executing `depositVeNFT()`.

If the intention is to only prevent a specific veNFT from migrating, it's fine. However, if the intention is to prevent the economic value or AERO principal represented by a restricted veNFT from being migrated, the restriction can be bypassed via various methods, including the following:

Method 1 - Using V2's merge:

- 1) Create a fresh veNFT containing 1 wei of AERO and make it permanent.
- 2) Unlock the restricted veNFT if it is permanent
- 3) Merge the restricted veNFT to the fresh veNFT, and the AERO will be transferred to the fresh veNFT
- 4) Deposit the fresh veNFT to the migration contract.

Method 2 - Use `split()` function to burns the restricted ID and creates two fresh IDs without propagating the restriction.

Method 3 - Use `withdraw` function. For an already-expired nonpermanent veNFT, `withdraw` burns the restricted ID and returns fungible AERO, which can immediately be submitted through `depositLiquid`

Impact

A holder can move a restricted veNFT's AERO into unrestricted token IDs through merge or split, or withdraw it as liquid AERO and use `depositLiquid()`. This defeats migration restrictions, allowing intentionally excluded principal to consume migration capacity and receive V3 conversion proceeds.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/migration/Migration.sol#L106>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/migration/Migration.sol#L55>

Tool Used

Manual Review

Recommendation

Define restrictions over the underlying economic value rather than only the veNFT ID, enforcing them within the migration contract at deposit time, since the v2 escrow's conversion paths are immutable. Decide explicitly how the untraceable liquid-deposit path is handled.

Discussion

rhollerhus

This one I remember we discussed on the call and will be acknowledged:

Acknowledged. It's a soft prevention technique, and it's understood that it can be bypassed.

Issue M-5: Refund Delay Lets Late Stakers Capture Undistributed Incentive Rewards [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/149>

Vulnerability Detail

In the original staker design, the staked positions keep accruing after `endTime`, but anyone (e.g., keeper) can settle the positions immediately at `endTime`. So the post-end accrual is bounded by how quickly someone acts. The new design introduces the `REFUND_DELAY` (hardcoded to 1 hour), which cause the permissionless settlement to be executable only after the `endTime + REFUND_DELAY`.

```
/// @inheritdoc ISlipstreamStaker
function unstakeToken(IncentiveKey memory key, uint256 tokenId) external override {
    Deposit memory deposit = deposits[tokenId];
    if (block.timestamp < key.endTime + REFUND_DELAY) {
        require(deposit.owner == msg.sender || _isApprovedForWithdrawal(deposit.owner,
            ↪ tokenId), 'not approved');
    }
}
```

Understood that the rationale of introducing the `REFUND_DELAY` is to stop a third party from force-slashing a stake whose min-stake window straddles `endTime`, causing the victim to be subjected to penalty. But the side effect is that for that hour a position keeps accruing and only its own owner can stop it. No one else can.

This allow malicious staker to stake at the last few seconds and become a major shareholder, and continue accrue tokens for ~3600 seconds after end time.

The share of the entire reward pot credited to the attacker's position at settlement depends on the incentive program length. The shorter the period, the greater the impact, as shown below Capture is $f * D / (T + D)$, where f is the attacker's share of the pool's total in-range liquidity (gauge-staked and idle LP included), D is the refund

delay, and T is the programme length.

Programme length	$f = 1.0$	$f = 0.9$
10 min	85.7%	77.1%
1 hour	50.0%	45.0%
4 hours	20.0%	18.0%
1 day	4.00%	3.60%
1 week	0.59%	0.53%
30 days	0.14%	0.12%

Note 1: The above figure assumes the attacker settles before the other LPs. If they all settle at `endTime`, the pot is consumed at the un-inflated denominator and the attacker gets almost nothing.

Note 2: Nothing is minted here. The value comes out of the refundee's undistributed remainder, or from LPs who settle later.

Impact

A staker can join shortly before `endTime`, monopolize in-range liquidity, and accrue rewards throughout the one-hour refund delay while third parties cannot unstake them. By settling first, they can capture a large share of the remaining reward pot, reducing refunds or rewards available to later-settling LPs.

Code Snippet

<https://github.com/dromos-labs/metadex-slipstream/blob/527d825a/contracts/stake/SlipstreamStaker.sol#L323-L327>

<https://github.com/dromos-labs/metadex-slipstream/blob/527d825a/contracts/stake/SlipstreamStaker.sol#L25>

Tool Used

Manual Review

Recommendation

Ensure the refund delay cannot increase a position's reward entitlement after the incentive ends. Preserve protection against premature forced settlement without allowing late stakers or settlement ordering to redistribute rewards from the refundee or other LPs.

Discussion

rholtarhus

Open PR here: <https://github.com/dromos-labs/metadex-slipstream/pull/51>

Issue M-6: Default Cap Changes Retroactively Reduce Accrued Gauge Rewards [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/152>

Vulnerability Detail

The Cap management section of the `gauge-factories.md` states:

"Unlike `setEmissionCap`, `setDefaultCap` does not settle gauges before writing – iterating over every gauge using the sentinel would be expensive, and the function is called rarely. Gauges using `DEFAULT_CAP_INDICATOR` pick up the new default at their next natural settlement, **briefly continuing to accrue at the old rate until then**; this is intentional and acceptable for an infrequent governance action."

However, this premise is not true. In the current design, the entire unsettled span is recomputed at the new cap the moment it settles.

As such, if the default cap is lowered, it is possible that the weeks of already-earned LP rewards get recomputed at a cap that did not exist while they were earned, leading to already-earned LP rewards to be permanently moved to the surplus bucket, which is unfair to the users.

When settling the unsettled span after the default cap get updated, it always read the live cap value and the walk has no idea a cap change happened.

```
function _settleGauge(address _gauge) internal {
    GaugeState memory _state = gaugeStates[_gauge];
    uint48 _to = lastSettlement;
    if (_to == _state.lastSettlement) return;

    uint128 _cap = _gauge == ZERO_GAUGE ? 0 : FACTORY_REGISTRY.emissionCap(_gauge);
    ↪ // Live Read
```

```
(uint128 _effectiveShare, uint128 _surplusDelta) = _walkGauge(_gauge, _cap,  
    ↪ _state, _to);
```

Impact

Lowering the default emission cap retroactively applies the new cap to the entire unsettled period. Rewards accrued under the previous cap can be permanently redirected to surplus, reducing LP compensation for past activity and violating the documented expectation that only future accrual is affected.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L1087-L1089>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/factories/GaugeFactory.sol#L152-L156>

Tool Used

Manual Review

Recommendation

Ensure default-cap changes preserve rewards accrued before the change and affect only subsequent emissions. The settlement model should maintain clear historical cap boundaries so governance updates cannot retroactively reprice previously earned LP rewards.

Discussion

agusduha

Fix in progress: <https://github.com/dromos-labs/metadex/pull/531>

Issue M-7: Delayed Deallocation Return Duplicates Cross-Chain Voting Power [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/153>

Vulnerability Detail

In PR404, it discusses the delayed-return retry problem where the return D already existed before emergency deallocation, leading to voting power being "double-count". Thus, it implements a clamp (`credit = min(D.amount, current root booking)`) in the `_creditDeallocation()` function to mitigate this.

However, the clamp doesn't cover all scenarios. There is an edge case where double-counting of VP still occurs in a similar situation.

1. Alice allocated 100 VP to Chain C by calling `allocateChains(T, [{Chain_C, +100}])`. State=[Root-CHAIN0=0, Root-Chain-C=100VP, Leaf-Chain-C=100VP, In-Flight-messages=[]]
2. Some time later, Alice decided to move 100 VP off Chain C. Thus, she executes `allocateGauges(T, Chain_C, [{DEALLOC_GAUGE, 100}])`, and an exit request called G is dispatched from Root -> Leaf. State=[Root-CHAIN0=0, Root-Chain-C=100VP, Leaf-Chain-C=100VP, In-Flight-messages={G}]
3. The governance decided to suspend C due to an emergency. The admin followed the documented procedure, checked whether any leaf Deallocate was in flight, and found none. Note that G is not the Deallocate message. Deallocate message will only be created when G landed on the leaf and gets processed. Then, the Deallocate message will be created from Leaf -> Root.
4. The governance calls `setEmergencyDeallocationAllowed(true)`.
5. Alice calls `emergencyDeallocate(T, Chain_C)`, and the `_creditDeallocation(Chain_C, T, 100)` credits the whole booking to CHAIN0. Message E is dispatched from Root->Leaf to unwind the 100 VP. State=[Root-CHAIN0=100VP, Root-Chain-C=0, Leaf-Chain-C=100VP, In-Flight-messages={G, E}]

6. G lands on the suspended leaf. The leaf drops 100 (`LeafVoter.sol:828`) and dispatches the return D. `State=[Root-CHAIN0=100VP, Root-Chain-C=0, Leaf-Chain-C=0VP, In-Flight-messages={E, D}]`
7. E lands on the suspended leaf, and it's a no-op since `Leaf-Chain-C` is already 0VP. The governance then unsuspends Chain C. `State=[Root-CHAIN0=100VP, Root-Chain-C=0, Leaf-Chain-C=0VP, In-Flight-messages={D}]`
8. Alice reallocates Chain C by calling `allocateChains(T, [{Chain_C, +100}])`. `State=[Root-CHAIN0=0, Root-Chain-C=100VP, Leaf-Chain-C=100VP, In-Flight-messages={D}]`
9. D is finally delivered. `_credit = min(100, 100) = 100`, so Root moves 100 off Chain C and back to `CHAIN0`. `State=[Root-CHAIN0=100VP, Root-Chain-C=0VP, Leaf-Chain-C=100VP, In-Flight-messages={}]`

In the end, the leaf (Chain C) holds 100 VP, but the root books 0 for the leaf (Chain C). That 100 is free again on `CHAIN0`, and can be reallocated to another chain (e.g., Chain D). So, a single stake of 100 directs 200 VP across the protocol

Impact

A delayed deallocation return can erase a new root booking while the leaf retains its voting power. The same stake becomes available on `CHAIN0` for allocation elsewhere, allowing one position to direct voting power across multiple chains and distort governance outcomes.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/Voter.sol#L642-L649>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/Voter.sol#L444-L474>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L829-L860>

Tool Used

Manual Review

Recommendation

Ensure deallocation returns already superseded by an emergency deallocation cannot be applied a second time, while ordinary delayed returns continue to apply. Emergency deallocation, suspension, reallocation, and delayed message delivery must preserve a single authoritative voting-power balance across root and leaf chains.

Discussion

0xng

Addressed here: <https://github.com/dromos-labs/metadex/pull/504>

Similar to the other comment, this will be added to a formal document with procedures.

Issue M-8: Delayed Pre-Suspension Message Restores an Invalid Emission Rate [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/155>

Vulnerability Detail

Note for PR404. During suspension, the leaf emission is set to zero. However, the protocol never invalidates an authentic old message already in flight. Thus, once the leaf is active again, the message can reinstall the old, too-high `emissionsPerVP` and make leaf liabilities grow faster than the root redemption ceiling.

This issue does not appear to be the expected drift, as this issue is more of an old and invalid rate being applied at the wrong time after the rate is cleared.

The following is the scenario to illustrate the issue:

1. Total emissions are 1 TOKEN/second. When total voting power is 1,000 VP, the root sends a message containing a rate of 0.001 TOKEN per VP per second, but delivery is delayed.
2. The leaf is suspended, resetting its rate to 0.
3. During suspension, total voting power grows to 10,000 VP, so the correct rate falls to 0.0001 TOKEN per VP per second.
4. The leaf resumes, and the delayed pre-suspension message installs the old 0.001 rate.
5. For a leaf with 100 VP over one week:
6. Root backing at the correct rate: 6,048 TOKEN
7. Leaf rewards at the stale rate: 60,480 TOKEN
8. The leaf therefore creates 54,432 TOKEN more claims than the root backs, potentially causing later redemptions to fail.

Impact

A delayed message can restore an excessive emission rate after suspension, causing leaf rewards to exceed root backing. This undercollateralizes cross-chain liabilities and may cause later reward redemptions to fail.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L993-L995>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L352-L354>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/bridge/LeafMessageOrchestrator.sol#L213-L215>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/bridge/LeafMessageOrchestrator.sol#L252-L253>

Tool Used

Manual Review

Recommendation

Ensure a resumed chain cannot re-adopt an emission rate derived from state predating its suspension, while still honoring the allocation content of such messages. Ensure the chain reliably receives the current rate after resuming, so stale-rate protection cannot leave it emitting nothing.

Discussion

0xng

Addressed here: <https://github.com/dromos-labs/metadex/pull/505>

Note that the team is currently working in a formal document containing the different operations, and how to handle them correctly.

Issue M-9: Stale Post-Outage Timestamps Create Unbacked Emission Claims [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/156>

Vulnerability Detail

Dromos is deployed on many chains. There is a risk that a chain may experience an outage, causing leaf and root accounting to diverge. Consider the following scenario:

- 12:00: Optimism's sequencer goes offline. Its last block timestamp is 12:00:00.
- 12:05: Base is still running. Alice allocates 10% of voting power; Base starts counting her allocation from 12:05.
- 12:09: Optimism restarts. It produces its first catch-up block now, but its timestamp is only 12:00:02 because it continues from the last OP block.
- Alice's queued Hyperlane message executes in that catch-up block, so Optimism starts counting her allocation from 12:00:02, almost five minutes before Base.

Assume total emissions are 10 TOKEN/second. Alice's 10% allocation earns 1 TOKEN/second:

```
Timestamp difference = 12:05:00 - 12:00:02 = 298 seconds
```

```
Unbacked leaf entitlement  
= 298 seconds × 1 TOKEN/second  
= 298 TOKEN
```

Optimism creates claims for 298 TOKEN more than Base's redemption ceiling backs. Eventually, up to 298 TOKEN of receipts cannot be redeemed.

The above scenario doesn't apply to all chains. It depends on how the chain processes pending blocks after service resumes. For the OP stack, OP blocks must advance from the parent by the configured block interval. After an outage, the chain can therefore produce overdue blocks whose timestamps remain behind wall time.

However, for Ethereum, missed Ethereum slots remain empty. The next block belongs to the current slot, and its execution timestamp must equal that slot's time. So this issue doesn't affect Ethereum or other chains that behave similarly.

Impact

A recovering chain may process allocations using stale timestamps, causing leaf emissions to accrue earlier than the root chain recognizes. This accounting divergence creates unbacked receipt claims, potentially leaving users unable to redeem their full entitlement.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L383-L419>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L1006-L1008>

Tool Used

Manual Review

Recommendation

Anchor cross-chain emission accounting to a time reference supplied by the root chain, so delayed messages and recovering chains cannot create entitlements that begin earlier than the root recognizes. Prevent the divergence up front, as such leaf-root gaps cannot be reconciled after the fact.

Discussion

Oxng

Fixed here: <https://github.com/dromos-labs/metadex/pull/506>

Issue M-10: Delayed Notifications Can Erase Migration Fee Entitlements [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/157>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

Note about the migration. Velodrome's leaf-gauge fee recovery assumes the flushed fees are booked to a leaf epoch that ends before the epoch in which the migration veNFT's reset lands. However, nothing enforces this. The leaf books flushed fees to the epoch in which the NOTIFY message is delivered, and each leaf action (e.g., NOTIFY, WITHDRAW, GET_FEES) is an independent Hyperlane message with no sequential-processing guarantee.

So, a delayed NOTIFY can therefore push the accumulated fee backlog into the same epoch as the reset, permanently removing the migration's entitlement.

Consider the following scenario

- Migration veNFT holds 99/100 of a gauge's voting-reward weight. The gauge's pool has accrued 100,000 USDC of fees.
- Root `distribute()` runs in E2, but the NOTIFY only reaches the leaf in E3, so the fee flush it triggers is booked to E3. Note that rewards can only be claimed 1 week later at E4.
- In E3, the migration votes for the next batch. `vote()` calls `_reset()` first, which sends a WITHDRAW that zeroes the migration's balance checkpoint within E3.
- `earned()` pays each epoch using the last checkpoint in that epoch, so the migration's E3 balance is 0.
- In E4, the E3 reward becomes claimable, but migration's E3 balance is 0. It receives 0 instead of ~99,000 USDC.

- The remaining 1-weight voter may receive the full 100,000 USDC, or the funds remain stranded if no eligible weight remains.

Note that the loss is order-independent within the epoch. Any epoch containing both the booking and the reset is forfeited. Also, claiming before the re-vote does not avoid it as E3 is only claimable from E4, and a claim during E3 pays zero.

Impact

A delayed fee notification can book accrued fees in the same epoch as the migration veNFT's reset, causing its final checkpoint to be zero. The migration may lose nearly all earned fees, which can be redirected to other voters or become permanently stranded.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/migration/Migration.sol#L192-L199>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/migration/Migration.sol#L202-L223>

Tool Used

Manual Review

Recommendation

The v2 reward accounting is immutable, so prevent the loss through the migration's own sequencing: a reset must never be checkpointed into the same leaf epoch in which flushed fees are booked, regardless of message delays. Entitlements booked into a completed epoch are durable and need no further protection.

Discussion

rholtterhus

Acknowledged. This will need to be handled operationally via carefully sequencing the actions of the migration NFT

Issue M-11: Dust Swap Bypasses Top-of-Block Premium [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/158>

Vulnerability Detail

Metadex PR #414 introduces a new top-of-block premium intended to charge the first economically meaningful arbitrage trade, which is most likely to exploit a stale pool price and extract value from LPs.

However, a searcher can first execute a negligible dust swap to consume the premium at almost no cost, then perform the actual trade at the lower normal fee. So, this defeats the feature's economic purpose and materially reduces the fees returned to LPs.

Impact

A searcher can consume the premium with a negligible swap, then execute the profitable arbitrage at the normal fee. This defeats the mechanism's purpose and reduces fee revenue returned to liquidity providers.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/hooks/dynamic/DynamicSwapFeeHook.sol#L443-L508>

Tool Used

Manual Review

Recommendation

Ensure premium consumption depends on a trade's realized effect on the pool rather than its mere occurrence or declared size, so that neither negligible nor split

transactions can exhaust it before the value-extracting arbitrage pays it.

Discussion

agusduha

Fixing in <https://github.com/dromos-labs/metadex/pull/496>

Issue M-12: Donation Buffer Lets Reported Surplus Exceed Ceiling and Mint Beyond Emission Schedule [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/162>

Vulnerability Detail

The two PRs involved (both merged 2026-08-20, 91 minutes apart):

- #459 "feat: add surplus spend entrypoint": adds `spendSurplus`, a governance entrypoint that mints a chain's accumulated dead surplus (`reportedSurplus + cumulativeSuspendedSurplus`, net of prior spends) to a chosen recipient.
- #460 "feat: add donation buffer and tests": adds a permissionless per-chain `donate()` buffer of real TOKEN and reworks `processRedeem` into two legs, minting up to ceiling headroom and paying the remainder from the buffer, so redeems stuck past the ceiling can clear without minting.

#460 widened the redeem solvency check to include the donated buffer:

```
if (_reserved + _amount > _ceiling + _buffered) revert CeilingExceeded();
```

The same check also guards the surplus high-water-mark store a few lines below:

```
if (_newSurplusReported > _surplusStored) _chainState.reportedSurplus =  
↪ _newSurplusReported;
```

So a leaf surplus report can now be accepted past `ceiling - totalRedeemed`, which was impossible before #460. The pot #459 shows governance has no ceiling clamp:

```
// spendableSurplus, registered chains:  
_spendable = _state.reportedSurplus + _state.cumulativeSuspendedSurplus -  
↪ _state.surplusSpent;
```

so `spendSurplus` will mint that whole figure. The problem: the part of the surplus that was only accepted because donations stood behind it gets minted as fresh supply, yet the donation backing it is never consumed by the spend. It stays in the buffer, still payable to future redeems, so the same donated backing pays out twice.

Consider the following scenario:

Chain C at ceiling 1,000, `totalRedeemed` 600, `reportedSurplus` 300.

1. The leaf over-accrues (the known drift): its `surplusAccrued` hits 500. A 150 redeem reverts `CeilingExceeded` ($500 + 600 + 150 = 1,250 > 1,000$) with the receipt already burned. This is exactly the stuck-redeem case donations exist for.
2. Donors add 400 to C's buffer. Redelivery passes ($1,250 \leq 1,400$): minted 0, the 150 is paid from the buffer, and `reportedSurplus := 500` is stored, above ceiling - `totalRedeemed = 400`.
3. `spendableSurplus(C)` now shows 500, described by its `NatSpec` as value "no receipt holder can claim". Governance honestly spends it, minting 500.

Result: $600 + 500 = 1,100$ minted against 1,000 ever scheduled, a 100 over-emission. The double count, explicitly: of the stored 500 surplus, only 400 fits the chain's mint entitlement (ceiling 1,000 - `totalRedeemed` 600). The extra 100 was accepted by the gate only because donated tokens stood behind it.

Spending the pot minted that 100 as fresh TOKEN, but the donation backing it was not consumed: it is still in the buffer (250 left), payable to future redeems. The same 100 of donated backing therefore produces two payouts, one minted and one still payable, and the minted one is exactly the over-emission.

Impact

Fresh TOKEN is minted beyond the chain's scheduled accrual, breaking the core emission-conservation invariant (`total minted` $\leq$ `total scheduled emissions`) through its sole backstop.

No malicious actor is required because `spendableSurplus` and its `NatSpec` steer an honest governance into the breaking spend.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/7442fc87/V3/src/voter/Voter.sol#L455-L478>

<https://github.com/dromos-labs/metadex/blob/7442fc87/V3/src/voter/Voter.sol#L725-L733>

<https://github.com/dromos-labs/metadex/blob/7442fc87/V3/src/voter/Voter.sol#L672-L693>

<https://github.com/dromos-labs/metadex/blob/7442fc87/V3/src/voter/Voter.sol#L419-L428>

Tool Used

Manual Review

Recommendation

Keep the widened gate only for deciding whether a redeem can be paid out (minted plus paid from the buffer). When storing the surplus advance, clamp `reportedSurplus` to `ceiling - totalRedeemed` (or equivalently clamp the pot read by `spendableSurplus`), so the surplus pot can never exceed the chain's remaining mint entitlement and the conservation identity #459 relies on holds again.

Discussion

rholtarhus

This one was addressed in [PR 470](#)

It's a different approach than mentioned, it only allows the donation to cover the redeem

Oxng

Update, we changed the approach and fixed it with the clamp here:

<https://github.com/dromos-labs/metadex/pull/543>

Issue M-13: Voting weight added after a swap captures that swap's fees [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/163>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

In the `_notifyFeesAmount` function, the timestamp's one and only settlement slot will be consumed at the start of the function on line 70 below, even if the call credits zero fees to nobody (See Line 76 below)

```
File: FeeDistribution.sol
069:   function _notifyFeesAmount(uint256 _pending0, uint256 _pending1) internal
    ↪   returns (uint256, uint256) {
070:       lastFeeUpdate = block.timestamp; // gate closes here, unconditionally
071:
072:       /// @dev Skip when there are no pending fees, unless an active gauge has
    ↪   buffered residual left to flush
073:       uint256 _buffered0 = bufferedFees0;
074:       uint256 _buffered1 = bufferedFees1;
075:       if (_pending0 == 0 && _pending1 == 0) {
076:           if (_buffered0 == 0 && _buffered1 == 0) return (0, 0); // leave without
    ↪   settling anything
077:           if (IGaugeFactory(gaugeFactory).emissionCap(gauge) == 0) return (0, 0);
078:       }
```

There is a public, unauthenticated, unpriced function that can reach this function, allowing anyone to consume the settlement slot for the current timestamp, which in turn can be exploited by malicious user:

```
File: VotingRewardsManager.sol
091:   function advanceGlobalPoints() external nonReentrant { // no access control
    ↪   at all
```

```

092:     uint256 _globalCheckpointIndex = globalCheckpointIndex;
093:     if (_globalCheckpointIndex == 0) revert NoGlobalCheckpoint();
094:
095:     uint256 _lastTs = _globalRewardPointHistory[_globalCheckpointIndex].ts;
096:     if (block.timestamp < VelodromeTimeLibrary.epochNext(_lastTs)) revert
↪ TooSoon();
097:
098:     _notifyFeesAndAdvanceHistory(_lastTs); // _notifyFeesAmount(0, 0) -> gate
↪ closed

```

Consider the following scenario:

1. The attacker holds real staked voting power P as a permanent position. The target gauge has an incumbent decaying voter with weight D
2. The attacker sees an incoming swap (e.g., in the mempool), and front-runs it by calling the `advanceGlobalPoints()` function (tx 1). Pending fees are 0, so nothing is credited to anyone, but `lastFeeUpdate = T` is written first, so the timestamp's single settlement slot is now spent.
3. The victim's swap lands and pays fees into `PoolFees` and advances the fee index. The target gauge's `pendingFees()` jumps from 0 to F , where F is the target gauge's pro-rata share of the swap fees. Nothing in the swap path touches the rewards manager, so the stale gate stands.
4. For a root-local target gauge, the attacker backruns the swap by calling `Voter.allocateGauges()`. `RootLocalAdapter` delivers the allocation synchronously to `LeafVoter` in the same transaction, allowing it to reach `VotingRewardsManager.checkpoint()` at timestamp T . (Note: Remote-leaf allocations might be more difficult due to the asynchronous nature of cross-chain messaging)

Line 77 sees `lastFeeUpdate == block.timestamp`, so the F sitting in `pendingFees()` is never read. The function goes straight to installing weight P .

```

File: VotingRewardsManager.sol
68:     function checkpoint(
69:         uint256 _tokenId,
70:         uint128 _allocated, // ← the attacker's weight P
71:         uint48 _stakeEnd,
72:         bytes calldata _data

```

```

73:    ) external nonReentrant {
74:        if (msg.sender != voter) revert NotVoter();
75:
76:        /// @dev Process pending fees only once per timestamp
77:        if (lastFeeUpdate != block.timestamp) { // ← tx 1 already set this to T
78:            (uint256 _pending0, uint256 _pending1) = IGauge(gauge).pendingFees(); //
            ↪ ← so F is never read and never credited
79:            _notifyFeesAmount(_pending0, _pending1);
80:        }
81:
82:        /// @dev Reset the allocation if no weights are allocated, or if a
            ↪ non-permanent stake carries no weight.
83:        if (_allocated == 0 || (_stakeEnd != 0 && (_stakeEnd <= block.timestamp ||
            ↪ _allocated < MAXTIME))) {
84:            _reset(_tokenId);
85:        } else {
86:            _checkpoint(_tokenId, _allocated, _stakeEnd, _data); // execution lands
            ↪ here instead and install attacker's weight P
87:        }
88:    }

```

5. At any later timestamp, F is finally divided, and the divisor is now $P + D$. The attacker's own claim is enough to trigger it, and the attacker collects the full $P/(P+D)$ share of fees that were generated entirely before it entered.

Impact

A voter can capture a share of fee revenue generated before its allocation, directly reducing rewards owed to voters who were active when the fees accrued. The redirected amount can approach the target gauge's full fee delta when the attacker supplies most of the post-allocation voting weight.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/6afd4a7e9fb518c8af0742133135e1741546167f/V3/src/rewards/FeeDistribution.sol#L70>

Tool Used

Manual Review

Recommendation

For every allocation checkpoint, consider crediting any fees the gauge has accrued before writing the voter's new weight, so those fees go only to the voters who were already allocated when the fees were earned.

Discussion

rholtterhus

Good point I see what you mean. These are my initial thoughts:

- In order to make it work perfectly, it would require supporting multiple checkpoints at the same timestamp, or something similar. For example, imagine the order within one timestamp is: Fee 1, User A allocates, Fee 2, User B allocates. User A should receive Fee 2 but not Fee 1, while User B should receive neither. This requires separate accounting checkpoints within that timestamp. Right now, after the first settlement in the block in Fee 1, both depositors will get access to Fee 2 even if they deposit later.
- Supporting multiple checkpoints within a timestamp could introduce other issues.
- The allocator would be locked into the allocation for the cooldown period. So this strategy essentially lets them allocate slightly later in the same block than the actual cutoff for receiving the fee, but they are then locked into that allocation for the entire cooldown period anyway.
- This also seems less severe than something like LP fees, e.g. imagine a situation where someone receives fees for liquidity they never provided. Here, the allocation itself does not directly affect the swap that generated the fees (but it is indirectly linked because they direct emissions to the gauge and help attract the liquidity that supports trading)

Therefore I think it's okay to acknowledge, but I'd be open to arguments from anyone

simplyoptimistic

so this has been discussed before during implementation and we accept it for now.

as i believe voting cooldowns will likely remain above 30 / 60 minutes in the short to medium term (think next 12 months), and because most chains we deploy on have fast blocks or are moving to faster blocks, we think this is acceptable. if it turns out to be a big issue in the future, it is fixable when we release a new amm / reward contract

Issue M-14: Gauge Emission Caps Accrue During Suspension, Diverting Protocol Surplus to LPs [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/165>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

Assume the following scenario:

- Gauge cursor sits on week boundary B0
- Resume poke lands 4 days in (F)
- The gauge is settled 3 days later ()
- Gauge uncapped rate $r = 0.5 \text{ TOKEN/s}$, $\text{cap} = 0.05 \text{ TOKEN/s}$.

In the current implementation, if a segment mixes a frozen span F with a live span , it credits $\min(r \cdot , \text{cap} \cdot (F +))$ instead of $\min(r, \text{cap}) \cdot$.

```
File: LeafAllocationLibrary.sol
1352:  function _accrueGaugeSegment(
..SNIP..
1359:  ) private view returns (uint128 _effective, uint128 _surplus) {
1360:      (uint256 _indexDelta, uint256 _timeIndexDelta) =
↪ _projectedChainAt(_leafStorage, _segmentEnd);
1361:      _indexDelta -= _state.lastIndex; // CLOCK 1 - scalar-weighted: 0 across
↪ the frozen span
1362:      _timeIndexDelta -= _state.lastTimeIndex;
1363:      int128 _slope = _state.point.slope; // slope over the segment, before any
↪ boundary reduction
1364:      uint48 _duration = _segmentEnd - _state.lastSettlement; // CLOCK 2 - wall
↪ clock: F +
```

```
1365:      uint256 _capShare = _calculateCapShare(_duration, _cap);
```

File: LeafAllocationLibrary.sol

```
1463:  function _segmentAccrual(
  ..SNIP..
1470:  ) private pure returns (uint128 _effective, uint128 _surplus) {
1471:      // End weight against the plain accumulator, plus the decay add-back when
  ↪ the segment decays; a permanent
1472:      // stake (`slope == 0`) skips the second term.
1473:      uint256 _segmentAllocated = Math.mulDiv(_endWeight, _indexDelta,
  ↪ PRECISION);
  ..SNIP..
1480:      // A zero-length final segment gets a zero cap share and contributes
  ↪ nothing.
1481:      uint256 _segmentEffective = _segmentAllocated < _capShare ?
  ↪ _segmentAllocated : _capShare;
1482:      // The narrowing is safe under `_capShare`, but the excess can exceed
  ↪ uint128 on a gauge-weight desync, so
1483:      // it fails fast instead of truncating into `surplusAccrued`.
1484:      _effective = uint128(_segmentEffective); // = min(r·, cap·(F+)) - should
  ↪ be min(r,cap)·
1485:      _surplus = (_segmentAllocated - _segmentEffective).toUint128();
```

Thus, the amount credited to LP in the current implementation will be, as follows:

```
credit = min(r·, cap·(F+))
credit = min(129600 TOKEN, 30240 TOKEN)
credit = 30240 TOKEN
```

However, the correct amount credited should be:

```
credit = min(r,cap)·
credit = min(0.5,0.05)·259200s
credit = 0.05·259200s = 12960 TOKEN
```

So, an excess of 17280 TOKEN is booked to LPs, which is supposed to belong to the surplus.

```
excess = 30240 - 12960 = 17280 TOKEN
```

Visualize using the above diagram. The excess equals $\text{cap} * F$, which is the portion that should not be taken into account when computing the amount.

Impact

For the week-segment straddling any suspend, sunset, or resume boundary, every over-capped gauge is credited up to $\text{cap} * F$ above its cap (17,280 TOKEN in the scenario above). The excess is claimable by the gauge's LPs, while `surplusAccrued`, which feeds `reportedSurplus` and the `spendableSurplus` pot that `spendSurplus` mints from, falls short by the same amount.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/a69f876b/V3/src/libraries/LeafAllocationLibrary.sol#L1352-L1365>

<https://github.com/dromos-labs/metadex/blob/a69f876b/V3/src/libraries/LeafAllocationLibrary.sol#L1443-L1446>

<https://github.com/dromos-labs/metadex/blob/a69f876b/V3/src/libraries/LeafAllocationLibrary.sol#L1463-L1485>

<https://github.com/dromos-labs/metadex/blob/a69f876b/V3/src/libraries/LeafAllocationLibrary.sol#L1085-L1088>

<https://github.com/dromos-labs/metadex/blob/a69f876b/V3/src/libraries/LeafAllocationLibrary.sol#L1304-L1331>

Tool Used

Manual Review

Recommendation

Consider pricing the cap allowance on the same clock as the accrual, so masked seconds grant no allowance.

Discussion

rholtterhus

Good point. We will acknowledge this for the contest. Here is what I added as a known issue to the auditor-guide:

- If a gauge settlement spans a period where leaf emissions were frozen, such as while the chain was in a suspended state, the frozen seconds still count toward its cap allowance, over-crediting the gauge at the expense of surplus. Operationally settling all gauges during chain state transitions helps mitigate this.

Issue L-1: Contribution addition and removal don't cancel out each other exactly [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/143>

Vulnerability Detail

Ideally a contribution removal should remove from bias/slope exactly what was added earlier. But the codebase doesn't take care of the differences arising from rounding and adds contributions piecewise ($\text{floor}(\text{amount}/\text{MAXTIME})$) while performing a single aggregate removal ($\text{floor}(\text{total}/\text{MAXTIME})$) in `_applyChainAllocation`.

```
function _addChain0Contribution(uint256 _dst, uint128 _amount) internal {
    if (_amount == 0) return;

    bool _dstHasAllocated = _allocationChainIds[_dst].length() > 0;
    (, uint48 _dstStakeEnd) = _liveStake(_dst);
    if (_dstHasAllocated && tokenStates[_dst].lastStakeEnd != _dstStakeEnd) revert
    ↪ DstShapeStale();

    // @audit bias and slope computed from just the newly added amount

    (int128 _bias, int128 _slope, int128 _perm) = _contribution(_amount,
    ↪ _dstStakeEnd, uint48(block.timestamp));
    _applyChain0AndTotal(_bias, _slope, _perm, _dstStakeEnd);

    .....
}
```

```
function _swapContribution(
    Point storage _point,
    mapping(uint48 _expiry => int128 _slopeChange) storage _slopeChanges,
    uint128 _prev,
    uint128 _new,
```

```

uint48 _oldStakeEnd,
uint48 _veStakeEnd
) internal {
    // Clamp `_now` up to `_point.ts` so the swap cannot land before the resolved
    ↪ point.
    uint48 _now = uint48(block.timestamp);
    if (_now < _point.ts) _now = _point.ts;

    // @audit the total aggregate amount is removed
    // 1. Remove the old contribution at its own expiry, clearing that expiry's
    ↪ slope change.
    {
        (int128 _bias, int128 _slope, int128 _permanent) = _contribution(_prev,
        ↪ _oldStakeEnd, _now);
        _applyContribution(_point, _slopeChanges, -_bias, -_slope, -_permanent,
        ↪ _oldStakeEnd);
    }

    .....
}

```

This can cause each swap to over-remove up to $n-1$ units of bias/slope corrupting the decaying weight mechanism

Impact

The decay mechanism for computing the live weight gets corrupted

Code Snippet

- [Voter._addChain0Contribution](#) – V3/src/voter/Voter.sol#L1279-L1295
- [Voter._applyChainAllocation](#) – V3/src/voter/Voter.sol#L1110-L1153

Tool Used

Manual Review

Recommendation

Always incorporate new changes by using the full amounts similar to `_swapContribution`

Discussion

0xng

Fixed here: <https://github.com/dromos-labs/metadex/pull/481>

Issue L-2: Risk with approvals within the Metarouter

[ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/145>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Given that the Metarouter works with both ERC20s and ERC721s, there is a certain risk when it comes to approvals.

Vulnerability Detail

Note: The following issue is not currently exploitable in the code. This issue is written in order to make sure a further change (e.g. due to gas optimizations) does not accidentally introduce it, as the impact could be Critical.

Given that the Metarouter operates both with arbitrary ERC20s and ERC721s, something to be really careful about is the ERC20.approve calls, as they share the same function selector as the ERC721 approve. This can be chained with the fact, that if an address is set as approved for all, from an NFT owner, they can call approve for that NFT id and it would succeed. So basically if you have a situation where the user can call approve on an arbitrary ERC20, towards an arbitrary address, an attacker can use it to steal the NFTs from users who have given approvalForAll towards the metarouter. It would look something like this:

1. Whale has given metarouter approval for all their NFTs. Whale owns id = 1.
2. Attacker calls a function which gives arbitrary ERC20 approval towards arbitrary address. He effectively calls 'positionmanager.approve(addr, 1).
3. Since the attacker has given full approval to the router, the call succeeds, and now the malicious addr has approval to take the NFT.
4. In a separate call, it pulls it.

For example, in the CLPositionLib, although the approval is not towards an arbitrary address, but towards the actual position manager pool, attacker could create a custom pool with one of the tokens being the positionmanager itself, and they'd be almost be able to deposit another person's NFT in the pool - the only thing preventing it is that there is a forceApprove for 0 at the end of the call, which would always revert. This assumption saves the attack here, but the other problem is that this assumption is not always enforced.

```
(IMetarouter.BalanceSpend memory _spend, bool _payerIsUser) =  
    abi.decode(_payload, (IMetarouter.BalanceSpend, bool));  
address _lpToken = IV2Gauge(_gauge).stakingToken();  
uint256 _amount = FundsLib.fund(_lpToken, _spend, _payerIsUser);  
// `depositFor` pulls exactly `_amount`, so success consumes the allowance in full  
↳ and no reset is needed.  
IERC20(_lpToken).forceApprove(_gauge, _amount);  
IGauge(_gauge).depositFor(_amount, _owner);
```

When we deposit in a gauge, a custom gauge address would allow arbitrary LP token to be passed and the attack to be possible. Here we do not have that forceApprove for 0 at the end to save it - but thankfully we have a gauge validation requiring that it is registered within the Voter, so the attack also cannot be performed here.

Just pointing it out as something to be aware of, since we are not looking at the absolute final version of the code right now and a small code refactor which removes forceApprove 0 at the end, maybe for gas saving, or simply forgetting it, might not seem like a big deal, but it could actually have large impact.

Impact

Loss of funds

Code Snippet

Tool Used

Manual Review

Recommendation

Do not remove the 0-value force approvals. Do not remove the `validateGauge` check

Discussion

agusduha

We will keep track of this issue until the final commit to avoid removing those checks

Issue L-3: Migration Burns veNFTs Before Preserving Unclaimed Rebases [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/148>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

`Migration.depositVeNFT` irreversibly merges and burns the source V2 veNFT without claiming or checking its outstanding rebase. So, it is possible in an edge case that some rebase can become permanently unclaimable.

Assume the following setup:

- veNFT #1234: permanent, 50M AERO locked, not voting, approved to Migration. It holds 10% of total veAERO.
- User claims rebases at Wed 23:58 UTC, so the in-progress week isn't claimable yet, so nothing is added. Epoch flips Thu 00:00 UTC; the completed week owes #1234 ~83,750 AERO (10% of the 837,500 weekly rebase), fixed from its week-end voting power.

Consider the following two scenarios:

Scenario A - migration lands first

- 1) 00:00:10 – `Migration.depositVeNFT(1234, user)` executes. It reads `locked(1234).amount = 50M` calls `V2_ESCROW.merge(1234, MIGRATION_TOKEN_ID)`. #1234 veNFT is burned.
- 2) 00:00:20 – anyone calls `Minter.updatePeriod()`. Because the distributor calculates individual entitlements using historical week-end veNFT balances, #1234 veNFT still has ~83,750 AERO entitlement despite already having been burned.

Scenario B - `updatePeriod` lands first

1) 00:00:10 – `Minter.updatePeriod()` makes the 83,750 AERO claimable for #1234 veNFT.

2) 00:00:20 – the user's already-submitted `depositVeNFT(1234, user)` executes. Within the `depositVeNFT` function, it performs no claim, reads `locked = 50M`, and burns #1234 with the rebase still unclaimed.

If the user attempts to claim the rebase by calling `RewardsDistributor.claim(1234)`, it will revert because lock and owner have been zeroed.

Impact

Migrating a veNFT can permanently forfeit accrued but unclaimed AERO rebases because the source token is burned before its rewards are preserved. Users lose any entitlement that becomes claimable immediately before or after migration.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/migration/Migration.sol#L98-L137>

Tool Used

Manual Review

Recommendation

Require migration to settle all finalized rebase entitlements of the source veNFT before it is burned, since nothing remains claimable afterward. Ensure deposits cannot proceed while the v2 epoch is stale, so entitlements finalizing around an epoch flip are not silently lost.

Discussion

rhollerhus

I think we will acknowledge this one, at least for now. This can be avoided off-chain by making sure users claim their rebase before migrating if they have one, and being especially careful around epoch flips

CC @simplyoptimistic @airtoonricardo FYI

simplyoptimistic

this is a known issue. its the same with rewards, and its a known issue on the v2 system, so yeah we will acknowledge it and will not be fixing it.

Issue L-4: Hyperlane Adapter Rejects Relay-Supplied Return Funding [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/154>

Vulnerability Detail

The intended design per the comments is that the `route()` is payable so that value attached to destination `Mailbox.process()` can become leaf pre-funding, which is used for funding the Deallocation return Hyperlane message from leaf to root.

```
File: LeafMessageOrchestrator.sol
108:   function route(uint256 _originChainId, bytes calldata _payload) external
    ↪ payable {
    ..SNIP..
120:       // `payable` so a relayer can fund the deallocation return an
    ↪ `AllocateGauge` triggers: the value lands in the
121:       // pre-funding pool. Value on any other type just tops the pool up,
    ↪ recoverable with `withdrawNative`.
```

Assume that the required return quote is 0.010 ETH, but the leaf pre-funding pool balance only have 0.003 ETH, so it is insufficient. Relayer call the `Mailbox.process()` with 0.010 ETH attached, and Hyperlane will invoke the Dromos's `HyperlaneAdapter` with the relayer's `msg.value`. Since `HyperlaneAdapter.handle` is declared as `nonpayable`, Solidity rejects nonzero call value and revert, even though sufficient funding is provided by the relayer in the first place.

```
File: IMessageAdapter.sol
56:   function handle(uint32 _origin, bytes32 _sender, bytes calldata _message)
    ↪ external;

File: HyperlaneAdapter.sol
73:   function handle(uint32 _origin, bytes32 _sender, bytes calldata _message)
    ↪ external {
74:       if (msg.sender != address(MAILBOX)) revert CallerNotMailbox();
```

```

75:     if (_origin != REMOTE_DOMAIN_ID) revert UnauthorizedOrigin();
76:     bytes32 _remoteAdapter = remoteAdapter;
77:     if (_remoteAdapter == bytes32(0)) revert RemoteAdapterNotSet();
78:     if (_sender != _remoteAdapter) revert UnauthorizedSender();
79:
80:     ORCHESTRATOR.route(REMOTE_CHAIN_ID, _message);
81: }

```

Note that apart from the need to declare `IMessageAdapter.handle()` and `HyperlaneAdapter.handle()` as payable, the `ORCHESTRATOR.route` at Line 80 also need to forward the ETH (`ORCHESTRATOR.route{value: msg.value}()`) for this intended feature to work.

Impact

Relayer-supplied ETH attached to Hyperlane delivery is rejected by the nonpayable adapter and never reaches the leaf pre-funding pool. This breaks the documented funding path for deallocation returns, although direct pool top-ups remain available, so it does not create a protocol-wide denial of service.

Code Snippet

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/bridge/hyperlane/HyperlaneAdapter.sol#L73-L81>

Tool Used

Manual Review

Recommendation

Make the entire Hyperlane receive path consistently accept and forward attached native value to the leaf orchestrator, preserving the documented relayer-funded pre-funding behavior. Align the interface and implementations so value handling cannot be silently omitted.

Discussion

0xng

Fixed here: <https://github.com/dromos-labs/metadex/pull/507>

Issue L-5: Chain Pause Traps LP Principal Through Forced Emission Claim [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/159>

Vulnerability Detail

Based on the documentation, it appears that withdrawals should always be allowed, but that is not true.

Per specs/gauges/emission-distribution:

| ST-4 | Deposits and withdrawals are never blocked by emission state

Per docs/gauge/gauge.md

- Withdrawals always allowed – deposits and withdrawals are never blocked, regardless of emission rate or cap.

However, withdrawal is hard-coupled to minting. `Gauge.withdraw -> CLGauge._withdraw` calls `_claimPositions` unconditionally before returning the NFT, and `_claimPositions` ends by calling `voter.mintEmissions`, which is gated `chainIsActive` and reverts `ChainNotActive` on any non-Active status.

For example, an LP with a staked CL position and non-zero accrued emissions calls `withdraw` while the leaf chain is `Paused` or `Suspended`. `mintEmissions` reverts and unwinds the whole transaction, so the `nft.safeTransferFrom` that returns the position never executes.

`V2Gauge` has the identical coupling: `_withdraw -> _claimEmissions -> voter.mintEmissions`. Only positions with exactly zero accrued emissions, or those inside the 5-block full-penalty window, can exit.

Impact

LPs with accrued emissions cannot withdraw staked positions while a chain is `Paused` or `Suspended`. Principal, swap fees and rewards stay locked for the operator-controlled

window, with no exit path for CL positions.

Code Snippet

<https://github.com/dromos-labs/metadex-slipstream/blob/527d825a/contracts/gauge/CLGauge.sol#L280-L307>

<https://github.com/dromos-labs/metadex-slipstream/blob/527d825a/contracts/gauge/CLGauge.sol#L351-L356>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L175-L178>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/voter/LeafVoter.sol#L273-L296>

<https://github.com/dromos-labs/metadex/blob/972882c8/V3/src/gauges/V2Gauge.sol#L167-L210>

Tool Used

Manual Review

Recommendation

Confirm whether a non-Active chain is intended to block LP principal exit, or only emission issuance. If the latter, decouple principal return from the mint path so a withdrawal can complete without minting and the accrual stays claimable once the chain is Active again. Any change must cover both gauge types.

Discussion

agusduha

Fix in progress: <https://github.com/dromos-labs/metadex-slipstream/pull/54>

rhollerhus

And here is the open PR for metadex:

<https://github.com/dromos-labs/metadex/pull/542>

Issue L-6: Re-orgs may cause principal token to be un-initialised [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-dromos-phase-2-july-20th-2026/issues/161>

Vulnerability Detail

The principal and yield tokens are created by using Openzeppelin's `clone` function which doesn't ensure that the implementation address has code. Even when the implementation is deployed first, it is possible for a re-org to sequence the cloning transaction ahead of the implementation deployment. This will cause the created principal and yield tokens to have a no-code address as the initial implementation before the actual implementation is deployed, after which the contracts can be initialised by an attacker

```
* @dev Same as {xref-Clones-clone-address-}[clone], but with a `value` parameter to
↳ send native currency
  * to the new contract.
  *
  * WARNING: This function does not check if `implementation` has code. A clone
  ↳ that points to an address
  * without code cannot be initialized. Initialization calls may appear to be
  ↳ successful when, in reality, they
  * have no effect and leave the clone uninitialized, allowing a third party to
  ↳ initialize it later.
  *
  * NOTE: Using a non-zero value at creation will require the contract using this
  ↳ function (e.g. a factory)
  * to always have enough balance for new deployments. Consider exposing this
  ↳ function under a payable method.
  */
function clone(address implementation, uint256 value) internal returns (address
↳ instance) {
    if (address(this).balance < value) {
```

Impact

Users and bootstrap address can lose their deposits

Code Snippet

<https://github.com/dromos-labs/metadex/blob/4f2cd2f4605e5ad30e0af3fd677a0bef1610b93f/V3/src/relay/libraries/RelayInitLib.sol#L126-L130>

Tool Used

Manual Review

Recommendation

Ensure that relay's are only deployed after the implementation deployment transaction has finalized or ensure that the implementation address has non-zero code length

Discussion

0xng

There's an open PR containing many fixes for the Relay:

<https://github.com/dromos-labs/metadex/pull/487>.

This one was tackled here: <https://github.com/dromos-labs/metadex/commit/857ebdb316e6a9af4f1b76b23188d48d2c0d25e2>

10xhash

lgtn

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.